RF-3: Software Architecture Specifications - Proposed Software Architecture

Lost Grenadiers

GitErDone

Audrey Dickson, Kaden Langford, Andrew Swartz

Indiana University Southeast

CSCI P445: CAPSTONE PROJECT DESIGN

Dr. Ron Finkbine

October 17, 2025

I. Overview

a. Lost Grenadiers operates on a client-server architecture designed to deliver fast, secure, and reliable navigation for students at Indiana University Southeast through a web browser. The front end operates through most modern browsers, providing users with an interactive and mobile-friendly map interface. The interface is built using HTML, CSS, and JavaScript with the Leaflet.js library. Users can search for destinations, view their own location, and request routes across the campus. The back end is a Flask-based REST API that processes user requests. It validates input and communicates with the PostgreSQL database enhanced with PostGIS and pgRouting. The backend is hosted in a virtualized environment managed by the hypervisor Proxmox. All communication between the client and server is encrypted through HTTPS using SSL/TLS certificates provided by Let's Encrypt. The overall system supports outdoor navigation, primarily using GPS, and indoor navigation primarily using mapped floor plans.

II. Subsystem Decomposition

a. Frontend (UI/UX)

i. Displays the interactive map, collects user input (destination) and visually renders routes and navigation steps calculated from the database.

b. API Gateway (Flask Server)

i. Manages communication between the front end and the database. It validates requests, applies security checks, and formats all responses.

c. Routing Engine

i. Uses pgRouting to calculate optimal paths between nodes in the campus graph, accounting for accessibility and multi-floor transitions.

d. Database Manager

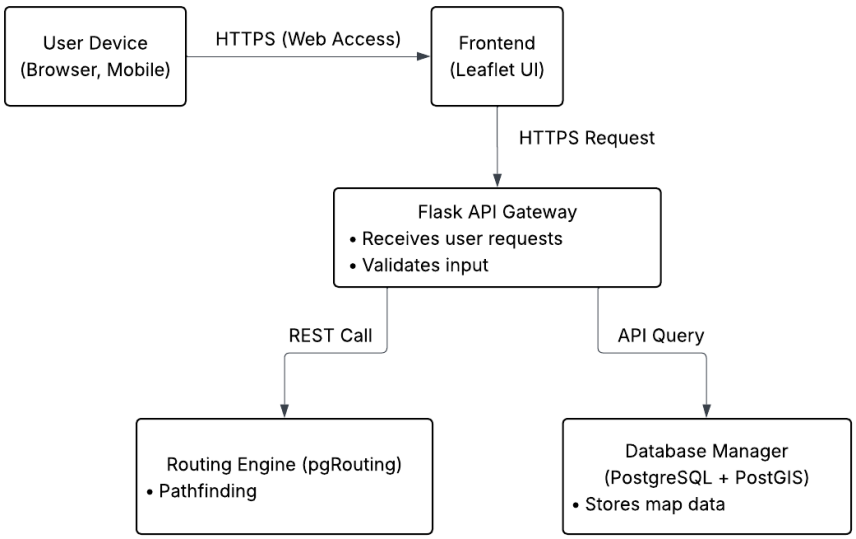
i. Stores and maintains spatial data for buildings, rooms, nodes, and edges. PostGIS enables spatial indexing and efficient queries.

e. Authentication and Security

i. Handles secure data transmission and input validation across the system. The Flask API enforces access restrictions by validating request origins and parameters, preventing unauthorized or malformed requests. Parameterized SQL queries are used throughout the back end to eliminate SQL injection risks. No login system will be implemented, but the configuration and deployment of the environment is limited to trusted network segments and controlled through firewall and router settings.

f. System Administration (DevOps)

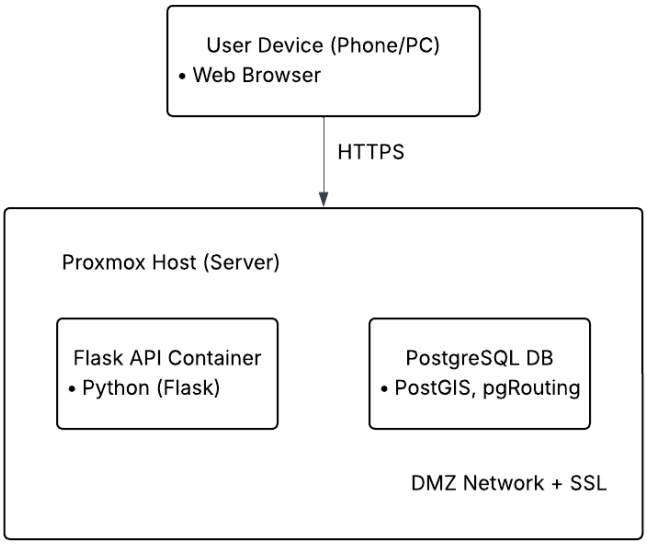
i. Oversees server deployment, backup configuration / scheduling, certificate renewals, network isolation (DMZ), and overall system health.



III. Hardware/software mapping

a. Virtualization

i. Lost Grenadiers is hosted on a Debian virtual machine within the hypervisor Proxmox. The virtual machine runs the Flask API and connects to the PostgreSQL instance.



b. Networking

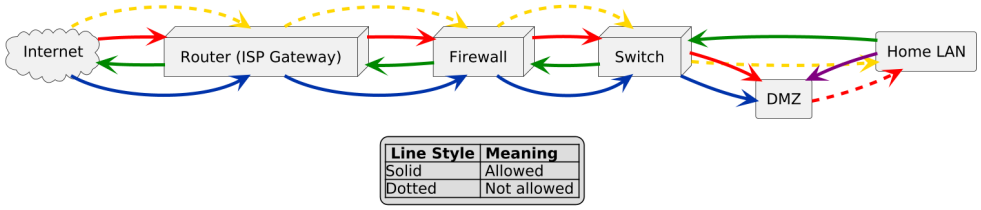
i. The front facing device of the home network is the router. This router will only allow port forwarding on 443.

ii. The firewall is placed after the router. This will route traffic between each vlan and will filter inbound and outbound traffic.

iii. A switch is placed after the firewall that distributes ethernet connections to different devices while enforcing vlan separation.

iv. The home LAN devices are freely plugged into their corresponding VLAN ports on the switch.

c. The Proxmox host is plugged into its corresponding DMZ VLAN port on the switch.



IV. Persistent Data Management

a. The Lost Grenadiers system stores detailed indoor navigation data for all campus buildings in a PostgreSQL database. The outdoor navigation is handled by Leaflet.js

b. Database Structure

i. **Buildings Table:** This table contains each building's code and name. Each building record references the floors that belong to it.

ii. **Floors Table:** This table contains every floor level for every building. Every floor record links back to its building and lists the rooms located on that floor.

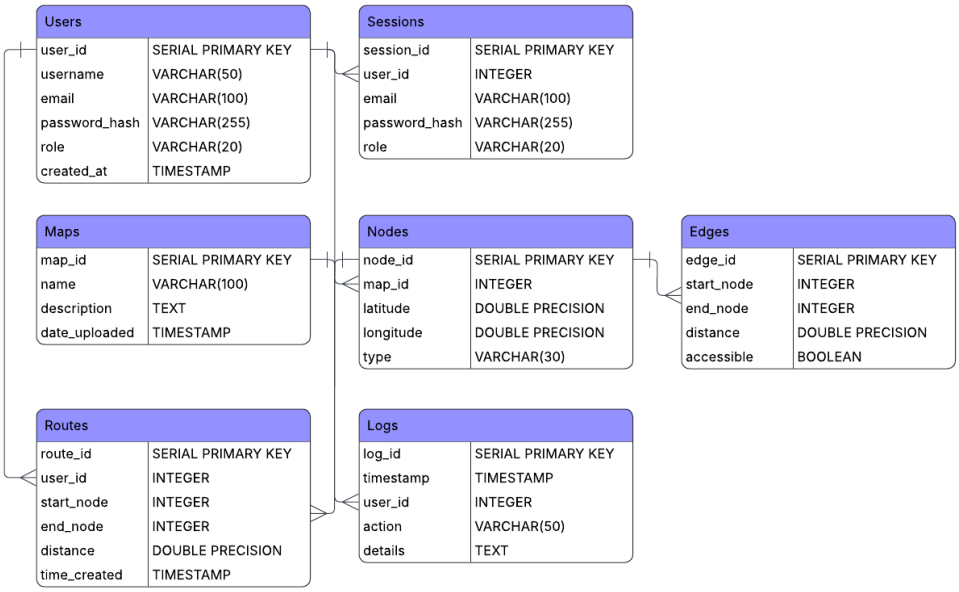
iii. **Nodes Table:** This table represents every navigable point in the building. This includes hallways, stairs, elevators, and rooms.

1. There is a column called "roomName" that will identify whether a point is a room or not a room. If the column value is populated, then it is a valid room. If the value is not populated, then it is assumed that this node is not a room. This means a result will never end on a node where the "roomName" column value is empty.
2. If the node is a staircase, then an accessibility Boolean column is marked false. This means the calculation of Accessibility routes is simply the shortest path algorithm that does not contain any nodes that are stair columns. It can be calculated figuratively by removing the staircase nodes from the search.
3. Staircases and elevators are also counted as nodes.

iv. **Edges Table:** This table defines the walkable connections between nodes. Each edge includes distance and accessibility information.

c. This structure allows the routing engine to compute efficient indoor paths by analyzing node and edge relationships while referencing building and floor data

for context. Weekly backups can easily be automated using pg_dump. Staff can bulk export/import data for server migration if needed.



V. Access Control and Security

a. Current Implementation (Project Development Phase)

i. Lost Grenadiers is hosted in a private Proxmox environment managed by our team. The back-end environment configuration is limited to our development team. The front-end environment is not currently restricted to anyone. Any end user is capable of navigating to our website, https://map.lostgrenadiers.org, and using our service. Our configuration of the website is accessible to users not affiliated with Indiana University Southeast. No user accounts or sensitive data exist within the system, so this exposure poses no confidentiality risk, but it increases availability and integrity risks. This implies there are potential security concerns such as:

1. **Internet attack**s: Public exposure could allow bots to probe the site for vulnerabilities or attempt denial-of-service attacks.
2. **Data Exposure:** Although no personal data is stored, improper query validation or improper logging could inadvertently leak structural or routing data.
3. **Network and Server Integrity:** Hosting outside Indiana University's managed infrastructure means reduced protection from enterprise-level firewalls, Intrusion Detection/Intrusion Prevention systems, and automatic patching.
4. **Physical Reliability:** Power loss, home ISP downtime, and IP address change could bring the service down. Power loss, home

ISP downtime, or IP address changes could cause temporary outages due to a lack of redundancy.

5. **Authentication / MFA:** Without IU's Duo integration, anyone on the internet who is not affiliated with Indiana University Southeast is capable of navigating to our website. Since campus maps are available online, this is not inherently a concern due to campus navigation information. This is more so a concern of potentially high amounts of wasted bandwidth for end users who do not genuinely need our service.

ii. While acceptable for testing, this configuration would not meet IU's long-term security or reliability standards, so migration to IU-managed infrastructure is essential for end-goal production.

b. Planned Implementation (Production Phase)

i. Lost Grenadiers is designed with a layered security model to ensure that access, data handling, and communications comply with Indiana University Southeast's authentication and IT security policies. The system uses IU's existing infrastructure for identity management and internal network protection.

ii. Authentication and Authorization

1. Access to Lost Grenadiers is restricted to Indiana University students, faculty, and staff. The application is hosted within the IU Southeast web ecosystem and is only accessible after users have successfully logged in through IU's login method. This ensures that only verified IU account holders can access navigation and routing features.

iii. Actors and Permissions

Actor	**Description**	**Permissions/Operations**
End User (Student, Faculty, or Staff)	Any user logged in through IU's authentication portal. Interacts with the map interface.	- Access the Lost Grenadiers map interface - View map and location - Search destinations - Request routes
System Administrator	Authorized IT personnel or project maintainers. Manages deployment, data updates, and security certificates.	- Start/stop services - Apply updates - Import/export data - Monitor logs
Developer	Authorized IU development team	- Push updates to backend/frontend repositories

	member. Maintains and updates codebase.	Deploy to test servers

iv. **Network Segmentation and Security**
   1. The Lost Grenadiers web application resides within the IU Southeast network and is not publicly indexed.
   2. The Flask API and PostgreSQL database operate on internal Proxmox virtual networks with no direct external exposure.
   3. Communication between front end, API, and database occurs over secured internal HTTPS connections.
   4. IU authentication tokens or session credentials are validated by the front-end layer before and API request is processed.
   5. Administrative SSH access is limited to IU IT network ranges and protected by a VPN and two-factor authentication.

v. **Data and Communication Security**
   1. **Transport Security:** All communication between client and server uses HTTPS with SSL/TLS certificates issued through IU Information Security Office or Let's Encrypt.
   2. **Session Security:** User authentication tokens from CAS/SSO are validated at session start; no credentials are stored locally by Lost Grenadiers.
   3. **Input Validation:** All API endpoints validate and sanitize parameters to prevent injection of malformed data.
   4. **Database Protection:** PostgreSQL connections are restricted to internal network traffic only, with role-based database permissions limiting modification rights.
   5. **Monitoring and Auditing:** Logs are automatically generated by Flask and PostgreSQL and retained according to IU data retention policy. Unusual activity triggers alerts for administrative review.

VI. Global Software Control

a. Lost Grenadiers is primarily event-driven, relying on client-side user actions and server-side REST API event responses.

b. Control Flow Type

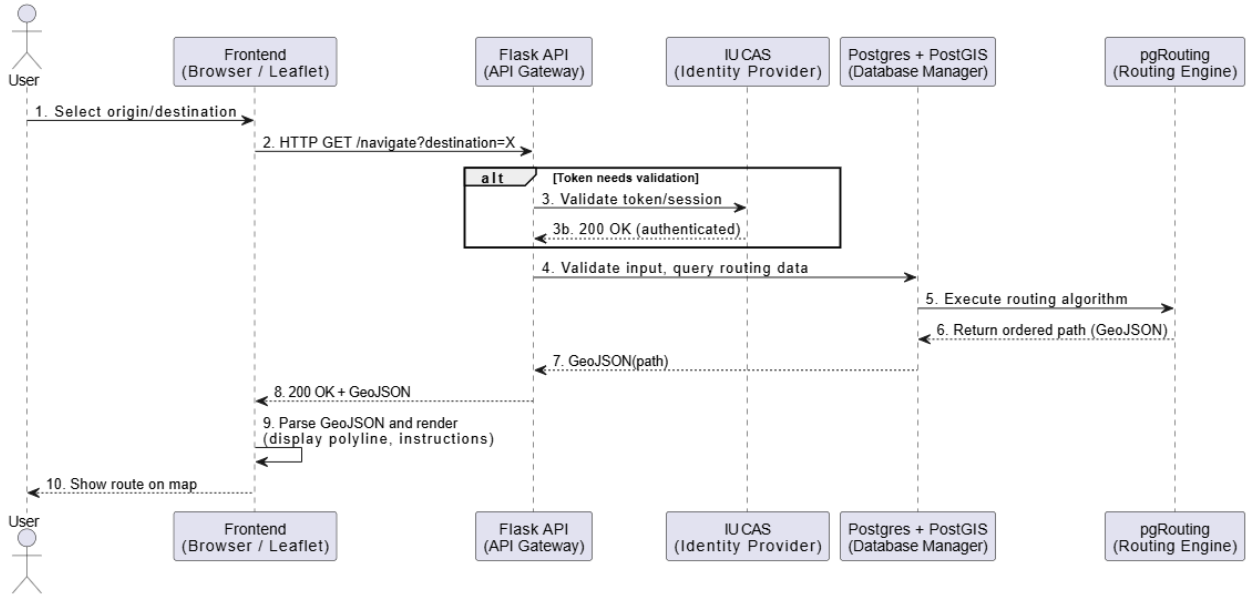
   i. Client-Side (Frontend): Event-driven; user clicks and map interactions trigger API requests.

   ii. **Server-Side (Flask API):** Request/response-driven; Flask listens for HTTP requests, processes them, queries the database, and sends JSON responses.

   iii. **Routing Engine:** Procedural; called synchronously during API request to compute optimal routes.

c. Control Flow Description

i. **Trigger:** User selects a destination or requests navigation.
ii. **Frontend:** JavaScript event captures request and sends HTTP GET/POST to Flask API.
iii. **API Gateway:** Flask validates input and forwards a SQL query to the database.
iv. **Routing Engine:** pgRouting computes optimal path between nodes.
v. **Database Manager:** Returns computed path as GeoJSON
vi. **API Gateway:** Sends formatted JSON response to the client.
vii. **Frontend:** Parses response and visualizes the route using Leaflet.js.



d. Threading and Concurrency

i. Flask handles concurrent requests using Gunicorn or uWSGI workers.
ii. PostgreSQL supports multiple concurrent queries, managed through connection pooling.

VII. Boundary Conditions

a. System Startup

i. System administrator starts the Proxmox virtual machine running Flask and PostgreSQL.
ii. Environment variables are loaded from .env files.
iii. Flask initializes routes and verifies connection to the database.

b. **Initialization**

i. Frontend fetches initial map tiles and building outlines from PostGIS.
ii. Cached data may be preloaded for performance.

c. **Shutdown**

i. Controlled shutdown through Proxmox.

   ii. Flask server accepting requests and flushes logs.
   iii. PostgreSQL safely closes all transactions and writes to disk.

d. Error and Exception Handling

   i. **Frontend:** Displays friendly messages for failed route requests.
   ii. **Backend:** Flask logs exceptions and returns appropriate HTTP status codes.
   iii. **Database:** Automatic rollback for failed queries.

e. Data Migration and Maintenance

   i. **Backup:** Weekly automated backup using pg_dump.
   ii. **Restoration:** Data can be bulk loaded using pg_restore or SQL scripts.
   iii. **Server Replacement:** Backup files and container configurations can be re-deployed to new Proxmox VMs.
   iv. **Regular Maintenance:** SSL certificates auto-renew monthly; OS and library updates performed quarterly.

Individual Contributions

Audrey Dickson

- Project management
- UI/UX design
- UI implementation

Kaden Langford

- Database schema design
- DevOps

Andrew Swartz

- Security Architect
- Backend Engineer
- Penetration Testing and Server Hardening

Key Personnel

Project Manager: Audrey Dickson

Lead Developer: Kaden Langford

UI/UX Designer: Audrey Dickson

QA/Tester: Andrew Swartz